

Monorepo 迁移方案

Monorepo 迁移方案

现状分析

- Next.js PWA: 根目录 src/, public/, next.config.ts 等
- 微信小程序: miniprogram/ (独立 tsconfig, 无 package.json)
- 共享代码: shared/ (types + constants), 在 miniprogram 里有一份完全相同的拷贝
- 构建: npm + husky pre-commit hooks
- 部署: GitHub Actions → SSH/SCP + PM2 (自有服务器)

目标结构



```

├── next.json
├── miniprogram.json
├── package.json          # @i-am-smart/tsconfig
├── eslint-config/       # 共享 ESLint 配置
│   ├── index.mjs       # 从根 eslint.config.mjs 提取
│   └── package.json     # @i-am-smart/eslint-config
├── turbo.json
├── pnpm-workspace.yaml
├── package.json          # 根: devDeps (turbo, typescript, pnpm)
├── .github/workflows/    # 更新 CI
└── docs/

```

迁移步骤

Phase 1: 基础设施（不动业务代码）

1. 切换到 pnpm

- 删除 package-lock.json
- 创建 pnpm-workspace.yaml
- 根 package.json 只保留 devDeps: turbo, typescript

2. 创建 turbo.json

```

{
  "$schema": "https://turbo.build/schema.json",
  "tasks": {
    "build": { "dependsOn": ["^build"], "outputs": [".next/**", "out/**", "dist/**"] },
    "dev": { "cache": false, "persistent": true },
    "lint": { "dependsOn": ["^build"] },
    "test": { "dependsOn": ["^build"] },
    "type-check": { "dependsOn": ["^build"] }
  }
}

```

3. 创建 packages/tsconfig/

- base.json: 从现有 tsconfig.json 提取公共配置
- next.json: Next.js 专用, extends base
- miniprogram.json: 小程序专用, extends base

4. 创建 packages/eslint-config/

- 从根 eslint.config.mjs 提取为可复用包

Phase 2: 提取共享包

5. 创建 packages/shared/

- 移动 shared/types/ → packages/shared/src/types/
- 移动 shared/constants/ → packages/shared/src/constants/
- 删除 miniprogram/shared/（改为依赖 @i-am-smart/shared）
- package.json name: @i-am-smart/shared

6. 创建 packages/api/

- 移动 src/lib/supabaseClient.ts → packages/api/src/
- 移动 src/lib/supabaseServer.ts → packages/api/src/

- 移动 `src/lib/supabaseAdmin.ts` → `packages/api/src/`
- PWA 中改为 `import { createClient } from '@i-am-smart/api'`
- 注意: `supabaseServer.ts` 依赖 `next/headers`, 只能在 Next.js 环境用, 需要分开导出

Phase 3: 搬迁 apps

7. 创建 **apps/pwa/**

- 移动: `src/, public/, supabase/, data/, scripts/`
- 移动配置: `next.config.ts, postcss.config.mjs, jest.config.ts, jest.setup.ts, components.json`
- 新建 `apps/pwa/package.json`: 继承现有依赖, 添加 `workspace` 引用
- `tsconfig.json` extends `@i-am-smart/tsconfig/next.json`
- 更新所有 `@/` 路径别名 (应该不需要改, 保持 `src/` 相对路径)

8. 搬迁 **apps/miniprogram/**

- 移动 `miniprogram/*` → `apps/miniprogram/`
- 新建 `package.json`, 依赖 `@i-am-smart/shared`
- 配置小程序 npm 构建: `miniprogram_npm` 需要 npm build 把 `node_modules` 里的包构建到小程序可用格式
- 更新 `project.config.json` 的 `miniprogramRoot` 如果需要

9. 创建 **apps/mobile/**

- `npm init @capacitor/app`
- `capacitor.config.ts` 中 `webDir`: `'../pwa/out'` (或 `.next` 取决于导出方式)
- `npx cap add ios`
- 不添加 android

Phase 4: 更新 CI 和工具链

10. 更新 **.github/workflows/deploy.yml**

- 改用 `pnpm`
- `turbo run build --filter=pwa` 构建 PWA
- 更新打包和 SSH/SCP 部署脚本路径

11. 更新 **.husky/ pre-commit hooks**

- 改为 `pnpm turbo run lint test --filter=...[HEAD]`

12. 更新 **Docker** 配置

- `docker/` 里的 `Dockerfile` 需要适配 monorepo 上下文

关键注意事项

小程序 npm 依赖

微信小程序不支持标准 `node_modules` 解析。方案: - `packages/shared/` 构建出 `dist/` (CommonJS 格式) - 小程序 `package.json` 引用 `@i-am-smart/shared` - 微信开发者工具 → 工具 → 构建 npm, 会把依赖打到 `miniprogram_npm/`

packages/api 的导出策略

`supabaseServer.ts` 依赖 `next/headers`, 不能被小程序或 mobile 使用:

```
// packages/api/package.json exports
{
  "exports": {
```

```
    "./client": "./src/supabaseClient.ts",  
    "./server": "./src/supabaseServer.ts",  
    "./admin": "./src/supabaseAdmin.ts"  
  }  
}
```

小程序和 mobile 只 import @i-am-smart/api/client。

部署

项目通过 GitHub Actions + SSH/SCP + PM2 部署到自有服务器，不使用 Vercel。monorepo 迁移后 CI 需要适配： - 改用 pnpm - turbo run build --filter=pwa 构建 PWA - 打包和部署脚本路径需要更新

路径别名

PWA 的 @/ 别名指向 apps/pwa/src/，搬迁后 tsconfig paths 不需要改，因为 src/ 的相对位置不变。

风险点

1. 小程序共享包构建 — 最容易出问题，需要验证 miniprogram_npm 构建流程
2. CI 适配 — deploy.yml 需要更新路径、pnpm 安装、turbo 构建命令
3. **import** 路径大量变更 — packages/api 提取后，PWA 里所有 @/lib/supabase* 引用都要改
4. 测试 — jest 配置需要适配 monorepo 路径解析

建议执行顺序

先做 Phase 1 + 3（基础设施 + 搬 PWA），确保 PWA 在 monorepo 下能 build + deploy，再做 Phase 2（提取共享包），最后 Phase 4（CI）。这样每一步都可以验证，降低风险。